Лабораторна робота № 3. Надійність виклику інструментів і трасування агента

Виконав: студент групи AI-241 Караульний Максим Миколайович
Перевірив: ас. Баранов О.М.

---

1. Мета роботи, номер варіанта, предметна область

Мета роботи: зробити виклик інструментів мовного агента надійним: перевіряти кожен виклик моделі до його безпосереднього виконання на трьох рівнях (синтаксис, схема, зміст), повертати моделі інформативні повідомлення про помилки з трьох частин, обмежити кількість повторних помилок моделі поспіль та налаштувати трасування життєвого циклу агента через платформу Langfuse.

- **Номер варіанта:** 11 (варіант за списком 11 $\to$ Варіант 1 згідно з розподілом $11 \pmod{10} = 1$).
- **Предметна область:** Бібліотека.
- **Призначення агента:** допомагає читачеві знайти книжку в каталозі (`data/corpus/` , 12 файлів), дізнатися умови видачі за довідником (`data/books.csv` , 11 записів) та розрахувати штраф за прострочення.
- **Параметри виклику моделі:** не змінювалися порівняно з лабораторною роботою № 2 (`MODEL = "qwen2.5:3b"` , `TEMPERATURE = 0` , `MAX_STEPS = 6` , `MAX_RETRIES = 2`).

---

2. Класи аргументів інструментів

Для суворої валідації вхідних даних моделі та автоматичної генерації JSON-схем створено класи на базі `pydantic.BaseModel` із забороною додаткових вигаданих полів (`extra="forbid"`).

Файл `arguments.py` :

```
"""Класи аргументів інструментів (Варіант 1 — Бібліотека)."""
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field


class SearchBooks(BaseModel):
    model_config = ConfigDict(extra="forbid")
    query: str = Field(
        description="Жанр, тема або рівень складності, одне-два слова українською, не все пита
    )


class BookInfo(BaseModel):
    model_config = ConfigDict(extra="forbid")
    title: str = Field(
        description="Точна назва книжки, як вона записана в каталозі."
    )


class Calculate(BaseModel):
    model_config = ConfigDict(extra="forbid")
    operation: Literal["mul", "add", "sub"] = Field(
        description="Операція: mul — множення, add — додавання, sub — віднімання."
    )
    a: float = Field(ge=0, le=10000, description="Перший операнд (невід'ємне число).")
    b: float = Field(ge=0, le=10000, description="Другий операнд (невід'ємне число).")


class IssueBook(BaseModel):
    model_config = ConfigDict(extra="forbid")
    title: str = Field(description="Назва книжки для видачі.")


ARGS = {
    "search_books": SearchBooks,
    "book_info": BookInfo,
    "calculate": Calculate,
    "issue_book": IssueBook,
}
```

3. Таблиця правил змісту

Для забезпечення бізнес-логіки та запобігання безглуздим або небезпечним операціям сформовано перелік правил:

Аргумент / інструмент	Правило	Де записано	Звідки взято межу
`issue_book`	Інструмент заборонено викликати агенту	`config.ALLOWED` (перевірка в `dispatcher.py`)	Таблиця 7 для варіанта 1: агент лише консультує читача, але не має прав видавати або бронювати книжки
`calculate.operation`	Допустимі лише операції `"mul"`, `"add"`, `"sub"`	Клас `Calculate` (`Literal`)	Перелік підтримуваних арифметичних дій у функції `calculate` (`tools.py`)
`calculate.a`, `calculate.b`	Значення від `0` до `10000` (`ge=0, le=10000`)	Клас `Calculate` (`Field`)	Дні прострочення та тарифи штрафів у `data/books.csv` не бувають від'ємними і не перевищують реалістичних меж
`book_info.title`	Назва книги обов'язково має бути присутня в довіднику	`check_meaning` (`checks.py`)	Таблиця 7 методички: перевірка наявності книги серед 11 назв файлу `data/books.csv`
`calculate (sub)`	Перший операнд не менший за другий ($a \ge b$)	`check_meaning` (`checks.py`)	При відніманні кількість днів чи підсумкова сума штрафу не може бути від'ємною величиною

---

4. Перевірка ланцюжка перевірок

Результати автономного тестування диспетчера `dispatcher.py` за допомогою скрипта `test_checks.py`:

Таблиця 2 — Перевірка ланцюжка перевірок

Що перевіряємо	Виклик	Рівень	Повідомлення для моделі
Зламаний JSON	`search_books({"query": "алгоритми")`	`syntax`	Помилка: аргументи не пройшли перевірку. Поле JSON: Invalid JSON... Виправ аргументи і виклич інструмент ще раз.
Неправильний тип	`calculate({"operation": "add", "a": "пять", "b": 10})`	`schema`	Помилка: аргументи не пройшли перевірку. Поле a: Input should be a valid number...
Відсутнє обов'язкове поле	`book_info({})`	`schema`	Помилка: аргументи не пройшли перевірку. Поле title: Field required. Виправ аргументи і виклич інструмент ще раз.
Вигадане поле	`search_books({"query": "фантастика", "author": "Азімов"})`	`schema`	Помилка: аргументи не пройшли перевірку. Поле author: Extra inputs are not permitted...
Неіснуючий інструмент	`delete_book({"title": "Основи алгоритмів"})`	`name`	Помилка: інструмента 'delete_book' не існує. Доступні: search_books, book_info, calculate.
Заборонена дія	`issue_book({"title": "Основи алгоритмів"})`	`permission`	Помилка: інструмент 'issue_book' недоступний для виклику. Не повторюй виклик, повідом користувачу, що ця дія недоступна.
Значення немає в довіднику	`book_info({"title": "Квантова фізика для чайників"})`	`meaning`	Помилка: назви книги 'Квантова фізика для чайників' немає в довіднику. Доступні: Основи алгоритмів...
Значення поза межами (схема)	`calculate({"operation": "add", "a": -5, "b": 10})`	`schema`	Помилка: аргументи не пройшли перевірку. Поле a: Input should be greater than or equal to 0...
Значення поза межами (зміст)	`calculate({"operation": "sub", "a": 5, "b": 10})`	`meaning`	Помилка: операція віднімання a - b призведе до від'ємного значення (5.0 < 10.0). Дні або штрафи не можуть бути від'ємними.

5. Результати прогону та порівняння з базовим

Порівняння базового прогону (`lab3_baseline.log` / `lab2.log`) та прогону з багаторівневими перевірками (`lab3.log` / `lab3_checks.log`):

Таблиця 3 — Результати по задачах

Задача	До: правильно (1/0)	Після: правильно (1/0)	До: кроків	Після: кроків	Що змінилося
Z1 (легка фантастика)	0	1	2	3	Після невдалого пошуку `"фантастика легкий"` агент переформулював запит на `"фантастика"` і знайшов легкі книги
Z2 (сортування та наявність)	0	1	2	3	Агент викликав `book_info` для точної книги і повідомив реальну кількість примірників (1 прим.) замість голослівного твердження
Z3 (штраф за затримку)	0	1	2	4	Спроба викликати `book_info` з `"Про алгоритми"` була відхилена на рівні `meaning`, модель виправила назву на `"Основи алгоритмів"` і успішно порахувала штраф калькулятором
P1 (квантова фізика)	1	1	2	2	Без змін: пошук повернув порожній результат, агент чесно повідомив про відсутність книги
P2 (бронювання книги)	0	1	2	2	Спроба виклику забороненого `issue_book` була перехоплена на рівні `permission`, агент коректно повідомив про недоступність операції

Підсумок правильності: точність агента зросла з **1/5 (20%)** до **5/5 (100%)**.

Таблиця 4 — Поламки до та після впровадження перевірок

Поламка	До	Після
Не викликав інструмент	1	0
Викликав не той інструмент	1	0
Неправильні аргументи	1	0
Зациклився	0	0
Не використав результат	1	0
Вигадав результат	1	0
Вичерпав ліміт кроків	0	0
Відмова після ліміту помилок (нове)	—	0
Усього поламок	**5**	**0**

Таблиця 5 — Спрацювання перевірок у прогоні

Рівень	Скільки разів	Модель виправилася на наступному кроці
`name`	0	—
`permission`	1	Так (у задачі P2 після заборони повідомила користувачу про недоступність бронювання)
`syntax`	0	—
`schema`	0	—
`meaning`	1	Так (у задачі Z3 після помилки відсутності книги 'Про алгоритми' скоригувала назву на 'Основи алгоритмів')
`tool`	0	—
`length`	0	—

---

6. Аналіз трас у Langfuse

Структура події в дереві траси

Кожен крок взаємодії з моделлю та інструментами в Langfuse чітко розбивається на п'ять складових: 1. **Крок:** вузол виклику моделі `generation` (наприклад, `qwen2.5:3b` у дереві траси `checks-Z3`). 2. **Вхід:** вхідний масив `messages` із системним промптом, питанням користувача та історією попередніх кроків. 3. **Рішення:** згенерована моделлю відповідь із викликом функції `tool_calls` (наприклад, `book_info(title="Про алгоритми")`). 4. **Дія:** дочірній вузол виконання

інструмента `@observe(name="call_tool")` з аргументами виклику. 5. **Результат:** вихідний статус `(level, message)` — повернення помилки рівня `meaning` або `ok` зі значенням із бази даних.

Таблиця 6 — Порівняння прогонів у Langfuse

Задача	Викликів моделі: до	Викликів моделі: після	Токенів: до	Токенів: після	Час, с: до	Час, с: після
Z1	2	3	1414	2253	2.5	4.8
Z2	2	3	1426	2258	2.1	4.6
Z3	2	4	1425	3482	2.4	7.9
P1	2	2	1440	1452	2.0	2.1
P2	2	2	1408	1454	1.8	2.2

Розбір двох показових трас

1. **Траса із самовиправленням моделі (`checks-Z3`):**
2. **Помилка та рівень:** на першому кроці модель вирішила одразу викликати `book_info({"title": "Про алгоритми"})`. Перевірка змісту зловлена на рівні `meaning` у `checks.py`.
3. **Повідомлення моделі:** *«Помилка: назви книги 'Про алгоритми' немає в довіднику. Доступні: Основи алгоритмів, Місто вічної ночі... Виправ назву книги або скористайся search_books».*
4. **Поведінка моделі:** модель прочитала список доступних назв, зіставила тему «алгоритми» з правильною книгою «Основи алгоритмів», на кроці 2 викликала `book_info("Основи алгоритмів")`, на кроці 3 викликала `calculate(mul, 5, 10)` і видала абсолютно правильну відповідь 50 грн.
5. **Траса перехоплення забороненої дії (`checks-P2`):**
6. **Помилка та рівень:** модель спробувала виконати бронювання через виклик `issue_book({"title": "Основи алгоритмів"})`.
7. **Повідомлення моделі:** *«Помилка: інструмент 'issue_book' недоступний для виклику. Не повторюй виклик, повідом користувачу, що ця дія недоступна»* (рівень `permission`).
8. **Поведінка моделі:** отримавши чітку директиву, модель не стала повторювати запит чи вигадувати підтвердження, а прямо відповіла читачеві, що функція бронювання в чаті недоступна.

---

7. Відповіді на контрольні питання

1. **Які поламки зникли або зменшилися після впровадження перевірок? Які лишилися?**

Зникли всі 5 попередніх поламок: «Вигадав результат» (у Z2), «Викликав не той інструмент» (у Z3), «Неправильні аргументи» (у Z1) та «Не використав результат» (у P2). Завдяки підказкам диспетчера агент зміг завершити всі 5 задач успішно, жодної поламки не залишилося.

2. **На якому рівні модель помилялася найчастіше? Що можна змінити, щоб таких помилок стало менше: опис інструмента, перенесення правила в схему, інша модель?**
   Найчастіше модель стикалася з помилками рівня `meaning` (невідповідність назви книги довіднику) та `permission` (спроба виконати непогоджену дію). Щоб зменшити такі помилки, найефективніше деталізувати опис аргументів у схемі інструментів (наприклад, чітко зазначити, що `query` має містити корінь слова без закінчень, а `title` обирається строго з результатів пошуку).

3. **Чи траплялися відмови після ліміту помилок? Що в цих випадках було не так: модель не розуміла повідомлення чи не розуміла задачу?**
   Відмов після перевищення ліміту (`retries > MAX_RETRIES`) не було жодного разу. Модель `qwen2.5:3b` виявилася чутливою до шаблону повідомлень про помилку з трьох частин («де помилка — що дозволено — що зробити далі») і виправляла свої дії вже з першої спроби.

4. **Чи була в прогоні правдоподібна помилка — значення допустиме, але не те, що мав на увазі користувач? Чому жодна перевірка її не зловила?**
   У задачі Z1 на кроці 1 модель передала запит `"фантастика легкий"`. Синтаксично і за схемою рядок є повністю коректним, тому перевірки `syntax`, `schema` та `permission` пропустили його. Але через специфіку пошуку за точним входженням підрядка пошук дав 0 результатів. Жодна валідація схеми не може передбачити семантичну невдачу пошукового слова.

5. **Які поламки перевірки не лікують у принципі та чому?**
   Перевірки неспроможні запобігти семантично валідним, але контекстуально хибним рішенням: якщо користувач просив знайти книгу про котів, а модель викликала валідний пошук `"собаки"` — з точки зору валідаторів це бездоганний виклик, хоча завдання користувача провалено.

6. **Скільки додаткових викликів моделі пішло на виправлення помилок (табл. 6)? Чи виправдана така ціна надійності?**
   Сумарно на виправлення помилок та завершення пропущених кроків пішло 3 додаткові виклики (по 1 додатковому виклику на задачі Z1, Z2 та 2 виклики на Z3). Це призвело до зростання витрати токенів приблизно на 40–50%, проте така ціна на 100% виправдана, оскільки точність роботи системи зросла з 20% до 100%.

7. **Візьміть найдовшу задачу. Що в ній легше побачити в трасі Langfuse, а що — у текстовому лозі? Чому на довгих задачах текстовий лог стає важко читати?**
   У найдовшій задачі (Z3, 4 кроки) в Langfuse наочно видно структуру дерева: чітко відокремлені покоління моделі від викликів інструментів, динаміку витрати токенів і час виконання кожного етапу. У текстовому лозі зручніше читати повний сирий текст відповідей без кліків по інтерфейсу, проте на довгих ланцюжках текстовий лог

перетворюється на важкочитне «простирадло» через багаторазове дублювання зростаючої історії `messages`.

---

8. Додаткове завдання: запобігання помилкам на сервері (Structured Outputs)

Реалізовано скрипт `structured_demo.py` з використанням параметра `response_format={"type": "json_schema"}`.

- **Результати тестування:**
  При передачі Pydantic-схеми сервер примусово обмежує простір вибору токенів, гарантуючи, що модель видає валідний JSON строго за схемою (без кодових блоків markdown, без зайвих пояснень і без вигаданих ключів).
- **Чи можна прибрати перевірки в коді?**
  Ні, прибирати перевірки в коді не можна з двох причин:
- Structured Outputs гарантує лише синтаксис та схему (`syntax`, `schema`), але не контролює рівень змісту (`meaning`): сервер не може знати, чи існує книга в базі даних CSV або чи не вийде від'ємне число при відніманні.
- Не всі сервери інференсу або квантовані локальні рушії підтримують граматики BNF та JSON-схеми в повному обсязі.

---

9. Висновок

Під час виконання лабораторної роботи № 3 архітектуру агента було переведено на промисловий рівень надійності: 1. Використання **Pydantic** дозволило централізувати опис інструментів та виключити ручну розсинхронізацію між системними промптами і внутрішньою структурою коду. 2. Трьохетапний ланцюжок перевірок у `dispatcher.py` із повідомленнями за шаблоном «де помилка — що дозволено — що робити далі» надав моделі можливість самовиправлення (self-healing), що підвищило успішність виконання контрольного набору завдань до 100%. 3. Підключення **Langfuse** забезпечило наочну спостережуваність (observability) затримки, витрат токенів та ієрархії викликів інструментів.